University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

AICOR Institute for Artificial Intelligence

Solution: Exercises for the Lecture Advanced Methods of AI

Asking, Describing, Constraining, Inferring

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

For the tutor: the 90 minutes

Time	Part	Students	Tutor; the sentence that leads on
0–8	Slides	open the notebook and start its setup cell, then listen	slides 1–8. *"Let us look at the world the robot knows."*
8–12	Prologue	warm-up on paper, in pairs	collect domain and extensions (2 min). *"The chef has a first question."*
12–34	Act 1 Asking	1(a) paper, then 1(b) predict and run, the vegetarian query, 1(c)	stop after 1(b) for vacuous truth and the closed world (3 min). *"All fitting objects is not always what we want: a command means one."*
34–44	Act 2 Describing	2(a) predict and run, 2(b)	ask: what should the robot do with 0 and with 2? *"Some descriptions must never fit anything."*
44–58	Act 3 Constraining	3(a) table and `allergens`, `IsSafeFor`, then 3(b) on paper, `dangerous`	draw ¬∀ ≡ ∃¬ on the board; one pair shows its red line. *"Instead of checking every time, the kitchen writes conclusions down."*
58–75	Act 4 Inferring	4(a), 4(b) predict, 4(c) predict and run	4(c) is the climax: let three pairs answer one of the three sources each. *"So far, every answer already existed. Dana orders one that does not."*
75–85	Act 5 Generating	5(a) predict and run, 5(b) on paper	collect 5(b) on the board: several answers are right. *"Let us put the evening together."*
85–90	Epilogue	5(c) summary table	show Figure 13; take-home message.

If time runs short, skip the think questions of Acts 2 and 3 and assign 5(b) for home. Pairs that are fast continue with the think questions; nobody waits. Every notebook cell runs as it is, so a pair that is stuck on a `TODO` can go on and compare with the solution later.

Prologue *Thursday evening, 19:30* 4 min

A small pizzeria in Bremen. A service robot works between the counter and the tables. Seven pizzas are on the counter, three tables have ordered, and two guests must not eat certain things. The robot knows all of this as objects: the classes of `pizza.owl` from last week, plus guests and orders.

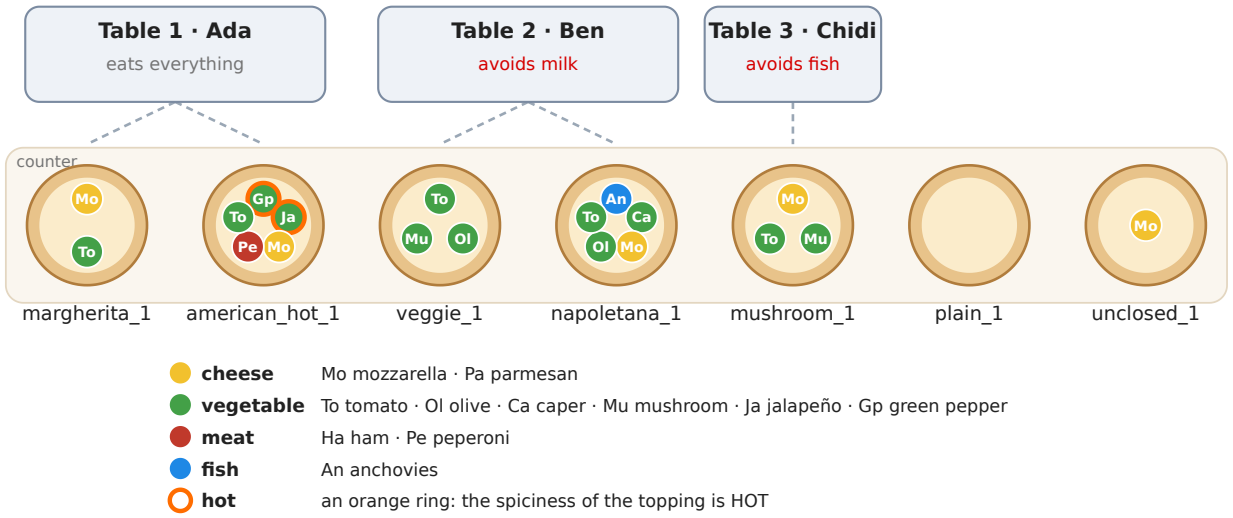


Figure 1. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered.

PAPER **Warm-up.** Figure 1 is an *interpretation* of first-order logic. What is its domain? What is the extension of the predicate `CheeseTopping`, and that of the relation `hasTopping`?

Solution

The domain is the set of all objects: 7 pizzas, their 19 toppings, 3 guests and 3 orders (the spiciness values and allergens are named by constants). The extension of CheeseTopping is the set of the 5 yellow dots, one mozzarella on each cheesy pizza; that of hasTopping is the set of pairs (pizza, dot on that pizza), 19 pairs. Every query today is evaluated in this *one* interpretation.

EQL, the Entity Query Language

EQL is part of the library KRROOD. Its queries are first-order formulas written over Python objects. The central expression is a **description**: a class and a pattern of attributes.

EQL	read it as	first-order logic
`a(Pizza)`	some object of class Pizza	a variable x with Pizza(x)
`a(Pizza)(toppings=a(CheeseTopping)())`	a pizza with a cheese topping	Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t))
`a(Pizza)(toppings=[A, B])`	whose topping list is [A, B]	the whole list: exactly these, in this order
`a(PizzaTopping)(spiciness=Spiciness.HOT)`	a topping that is hot	spiciness(t) = hot
`.from_(pizzas)`	… among these objects	the range of x
`the(Pizza)(…)`	*the* pizza that …	exactly one object fits
`topping = flat_variable(pizza.toppings)`	a topping of that pizza	a variable t with hasTopping(x, t)
`HasType(topping, VegetableTopping)`	it is a vegetable topping	VegetableTopping(t)
`exists(t, φ)`, `for_all(t, φ)`	some, every topping …	∃t φ, ∀t φ, over the toppings of x
`not_(φ)`, `or_(φ, ψ)`, `.where(φ, ψ)`	not, or, and	¬φ, φ ∨ ψ, φ ∧ ψ
`set_of(order.guest, pizza)`	return these together	the free variables of the formula
`.tolist()`	evaluate now	all assignments that satisfy it

Writing a query computes nothing; it builds a symbolic expression, evaluated only by `.tolist()`. Every `a(…)` introduces a variable of its own: two `a(Pizza)` are two pizzas, as x and y are two variables.

What the robot *does* with a description is what this tutorial is about. There are five uses, one per act:

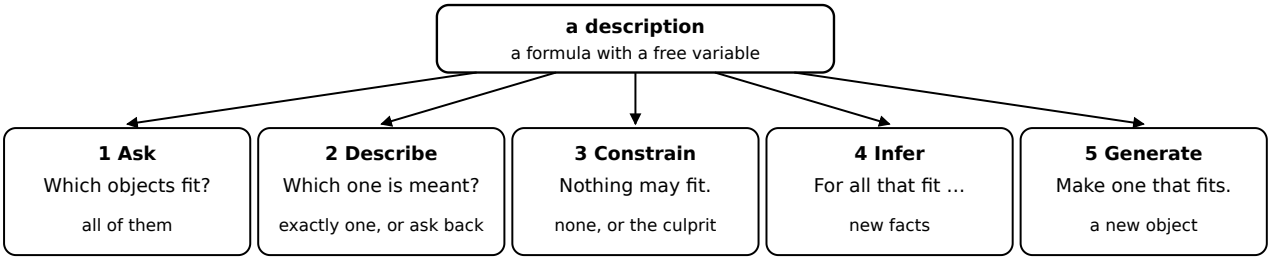


Figure 2. One description, five uses. The expression stays the same; what changes is where the candidates come from and what is done with those that fit.

Act 1 · Asking *"Which pizzas have cheese?"* 22 min

The chef calls across the kitchen. The cheesy pizzas go back into the oven for a minute, so that the cheese melts. The robot has to find them.

(a) A description is a formula. PAPER Write `a(Pizza)(toppings=a(CheeseTopping)())` as a first-order formula with the free variable x, and ring the pizzas of Figure 1 that satisfy it. NOTEBOOK Run 1(a) and compare.

Solution

Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)). The free variable is what is asked for, the bound variable t is a witness. Answers: margherita_1, napoletana_1, american_hot_1, mushroom_1, unclosed_1 (Figure 3). A pattern given for the list `toppings` describes one element: it is the ∃.

The query computes the set of objects that satisfy the formula *in one interpretation*: it checks a model. It does not ask whether the formula follows from a theory, that is, whether it is true in *every* model.

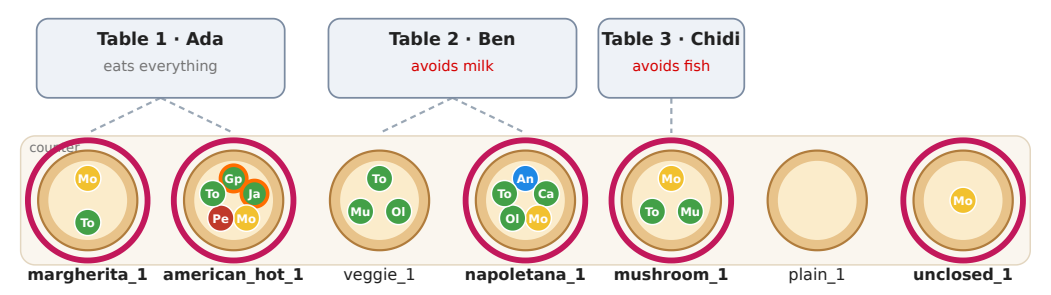


Figure 3. The answers of 1(a), ringed.

(b) Some, only, none. A pattern can only say *some topping is* …. For *every* and *no*, the topping gets a name and is quantified in `.where(…)`. Each query introduces its own variables `pizza = a(Pizza).from_(pizzas)` and `topping = flat_variable(pizza.toppings)`:

```
Q1 = pizza.where(for_all(topping, HasType(topping, VegetableTopping)))
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping)(), a(TomatoTopping)()]).from_(pizzas)
Q3 = pizza.where(not_(exists(topping, HasType(topping, CheeseTopping))))
```

PAPER Write each query as a formula. PREDICT Mark which pizzas each returns. Then run 1(b).

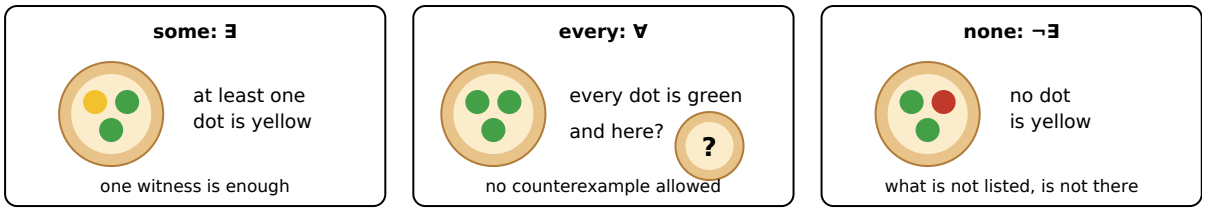


Figure 4. The three quantifier patterns, as the robot checks them on one pizza.

Solution

	margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed	formula
Q1					×	×		Pizza(x) ∧ ∀t (hasTopping(x, t) → VegetableTopping(t))
Q2	×							Pizza(x) ∧ ∃t_1∃t_2 (toppings(x) = [t_1, t_2] ∧ Mozzarella(t_1) ∧ Tomato(t_2))
Q3					×	×		Pizza(x) ∧ ¬∃t (hasTopping(x, t) ∧ CheeseTopping(t))

- **Q1:** plain_1 has no toppings, so the implication is true for every t: there is no counterexample (vacuous truth). It is p3 of Worksheet 2.
- **Q2:** a list of patterns describes the whole list. This implies the closure of Worksheet 2 (some mozzarella, some tomato, and nothing else) and is stronger: it also fixes the number and the order of the toppings. mushroom_1 has a third topping, unclosed_1 only one.
- **Q3:** nobody stated "veggie_1 has no cheese". The query concludes it because the stored list is taken to be complete: what is not stored is false (closed-world assumption).

> **Think.** Last week the OWL reasoner refused to classify `UnclosedPizza` (a pizza with some mozzarella) as vegetarian. NOTEBOOK Write the query `vegetarian` (no meat and no fish topping). Is unclosed_1 an answer? Who is right, the query or the reasoner?

Solution

```
vegetarian = pizza.where(not_(exists(topping, or_(HasType(topping, MeatTopping),
HasType(topping, FishTopping)))))
```

Answers: margherita_1, mushroom_1, veggie_1, plain_1, unclosed_1; in logic ¬∃t (hasTopping(x, t) ∧ (MeatTopping(t) ∨ FishTopping(t))).

Both are right; they answer different questions. The query evaluates the formula in the one interpretation the robot has, where the list of toppings is complete (closed world). The reasoner asks whether "vegetarian" *follows* from what was said, i.e. holds in every model of it; there are models in which the pizza also has ham (open world). A robot looking at a pizza usually wants the closed-world answer, provided its knowledge of the pizza is complete.

(c) Following the relations. *Ada at table 1 does not like spicy food: "Is anything hot on my table?"* Patterns nest as deep as the objects do. PAPER Write the orders that contain a pizza with a hot topping as a formula with the free variable *o*. NOTEBOOK Write `hot_pizzas` and `hot_orders`.

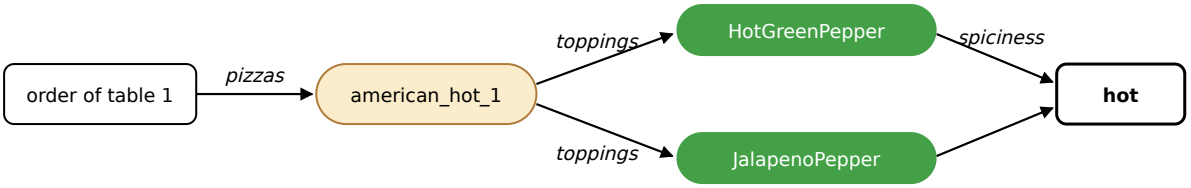


Figure 5. The paths from Ada's order to a hot topping. There are two.

PREDICT The notebook then asks for the witnesses themselves: `set_of(order.guest, pizza, topping).where(topping.spiciness == Spiciness.HOT)`. How many answers does it have, while `hot_orders` has one? _____

Solution

Order(o) ∧ ∃x (hasPizza(o, x) ∧ ∃t (hasTopping(x, t) ∧ spiciness(t) = hot))

```
hot_pizzas = a(Pizza)(toppings=a(PizzaTopping)(spiciness=Spiciness.HOT)).from_(pizzas)
hot_orders = an(Order)(pizzas=a(Pizza)(toppings=a(PizzaTopping)
(spiciness=Spiciness.HOT))).from_(orders)
```

Answers: american_hot_1, and the order of table 1. The `set_of` query has **two** answers, one per path of Figure 5. An answer is an **assignment** of values to the free variables that makes the formula true. In `hot_orders`, x and t are bound by ∃: only o is free, and the order is returned once, however many witnesses there are.

Act 2 · Describing *"Bring me the pizza with olives."* 10 min

Commands do not name objects; they describe them. Ben at table 2 waves at the robot.

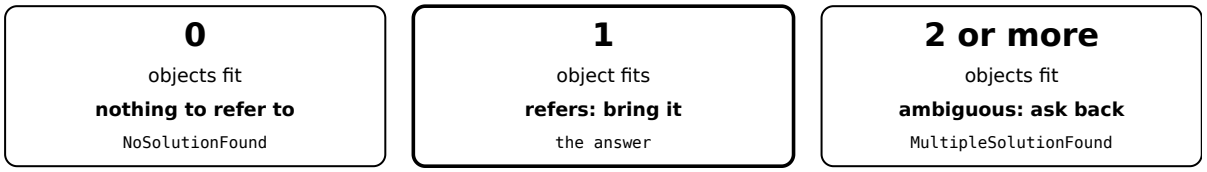


Figure 6. `a(…)` accepts every object that fits; `the(…)` claims that exactly one fits, and raises an exception otherwise.

(a) *a* or *the*? PREDICT For each command, how many pizzas fit, and what should the robot do? Then run 2(a).

Solution

Command	Pizzas that fit	What the robot does
"the pizza with ham"	none	reports that there is no such pizza (NoSolutionFound)
"the pizza with fish"	napoletana_1	brings it
"the pizza with olives"	napoletana_1, veggie_1	asks back (MultipleSolutionFound)

Russell analysed "the F is G" as ∃x (F(x) ∧ ∀y (F(y) → y = x) ∧ G(x)): there is exactly one F, and it is G. If there is no F or more than one, Russell calls the sentence false; Strawson argued that it is neither true nor false, because its presupposition fails. The robot, which refuses instead of answering, takes Strawson's side: picking one pizza at random would be a guess, not obedience.

(b) Making a description precise. The robot asks back: "The one with fish, or the one without?" Ben: "Without fish." PAPER Write the precise description as a formula. NOTEBOOK Add the condition to `the(Pizza)(toppings=a(OliveTopping)())` with `.where(…)`, so that `the` succeeds.

Solution

Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ OliveTopping(t)) ∧ ¬∃t' (hasTopping(x, t') ∧ FishTopping(t')); tonight only veggie_1 satisfies it.

```
pizza = the(Pizza)(toppings=a(OliveTopping)()).from_(pizzas)
topping = flat_variable(pizza.toppings)
olives_without_fish = pizza.where(not_(exists(topping, HasType(topping, FishTopping))))
```

Two quantifiers, two variables: the pattern names the olive, `topping` ranges over all toppings. A good question back names a property that one candidate has and the other has not.

> **Think.** "The pizza with cheese" does not refer tonight: five fit. Does "the pizza with cheese *of table 3*" refer? What does this tell you about `the`, and about how people understand "the …" in everyday speech?

Solution

Yes: mushroom_1 is the only cheesy pizza Chidi ordered. Whether a description refers depends on the **domain** it is evaluated in, not only on the description. People narrow the domain by context (the table, what is in sight, what was just said) until "the" works; a robot has to add that context to the description, for example as `.from_(order_of_table_3.pizzas)`.

Act 3 · Constraining *"Ben must never get milk."* 14 min

Ben avoids milk. Nowhere does it say "napoletana_1 contains milk": it has to be computed. Mozzarella is a cheese, cheese contains milk, and the pizza has that topping.

(a) Stored or computed? `symbolic_function` turns a Python function into a symbol of the query language, and a `Predicate` is a relation decided by a computation. PAPER For each fact below, is it stored in the objects (S) or computed (C)? NOTEBOOK Complete `allergens` and `IsSafeFor`.

Solution

hasTopping(x, t)	**guest(o) = g**	**∀t (Mozzarella(t) → Cheese(t))**	**milk ∈ allergens(x)**	**IsSafeFor(x, g)**
S: the list `toppings`	S: the field `guest`	S: the class hierarchy	C: `allergens`	C: the predicate

```
@symbolic_function
def allergens(pizza):
    found = {Allergen.GLUTEN}
    if any(isinstance(t, CheeseTopping) for t in pizza.toppings): found.add(Allergen.MILK)
    if any(isinstance(t, FishTopping) for t in pizza.toppings):   found.add(Allergen.FISH)
    return found

class IsSafeFor(Predicate):
    def __call__(self): return not (allergens(self.pizza) & self.guest.avoids)

pizza = a(Pizza).from_(pizzas)
safe_for_ben = pizza.where(IsSafeFor(pizza, ben))       # veggie_1, plain_1
```

In a query, stored and computed relations look the same. Giving a symbol a program as its meaning is called **procedural attachment**.

(b) **A constraint is a query that must have no answer.** The kitchen's rule: every pizza of an order is safe for the guest of that order. PAPER Write it as a formula (hasPizza, guest, IsSafeFor), and then its negation, pushed inwards.

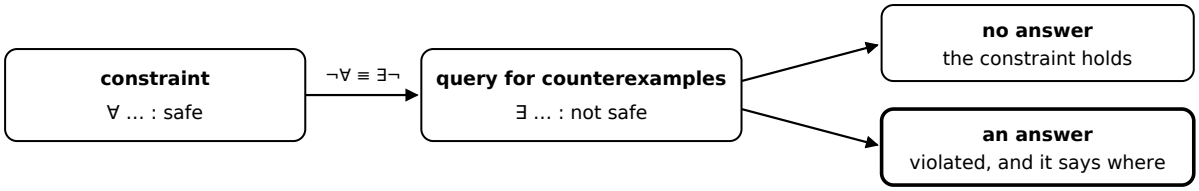


Figure 7. A universal statement is violated exactly when a counterexample exists.

PREDICT How many violations are there tonight? Draw each one as a line in Figure 1. NOTEBOOK Write `dangerous` and check.

Solution

Constraint: $\forall o\, \forall x$ (hasPizza(o, x) → IsSafeFor(x, guest(o))).
Negation: $\exists o\, \exists x$ (hasPizza(o, x) ∧ ¬IsSafeFor(x, guest(o))). As a query, o and x are left free, so that each answer names a culprit:

```
order = an(Order).from_(orders)
pizza = flat_variable(order.pizzas)
dangerous = set_of(order, pizza).where(not_(IsSafeFor(pizza, order.guest)))
```

One violation: table 2, Ben, napoletana_1 (mozzarella, hence milk). EQL could evaluate the ∀ directly, but the answer would only be “false”; the counterexample query says which plate not to serve.

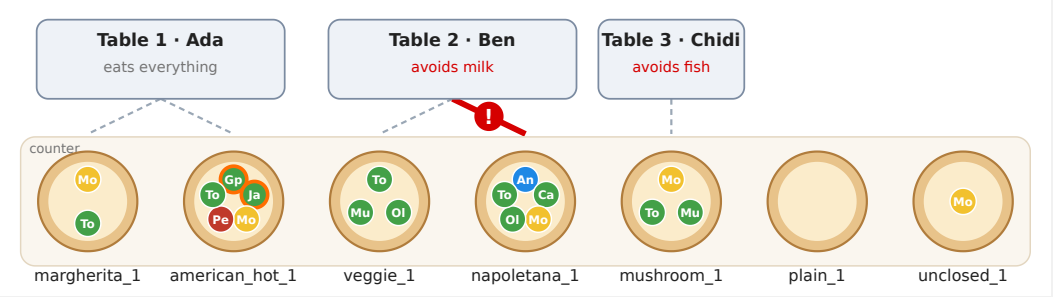


Figure 8. The one violation of the constraint.

> **Think.** Suppose the kitchen's rule were an axiom of last week's ontology. What would the reasoner report for tonight's facts? And if the mozzarella of the Napoletana had simply not been recorded?

Solution

An axiom does not check facts; it restricts the models. Facts that contradict it leave no model at all: the ontology is **inconsistent**, and from an inconsistent theory every formula follows. The reasoner reports the inconsistency, not a plate. If the mozzarella had not been recorded, there would be no contradiction: in the open world, the reasoner would rather conclude that the Napoletana is safe for Ben, because the axiom demands it. Used as an **integrity constraint**, the same formula is checked against the one interpretation; the knowledge base stays usable, and the answer is the culprit.

Act 4 · Inferring *The kitchen writes warnings and labels.* 17 min

The menu must list the allergens of every pizza, and a label: vegetarian, vegan or meaty. Instead of computing these for every question, the kitchen writes them down once, as new facts.

(a) A rule creates facts. `inference(AllergenWarning)(pizza=D, allergen=Allergen.MILK)` creates one new warning object for every pizza that fits the description D. With D the description of Act 1(a), it is the rule

```
∀x (Pizza(x) ∧ ∃t (hasTopping(x, t) ∧ CheeseTopping(t)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w) = milk))
```

PAPER Which part of the formula is D, and which part does the rule build? PREDICT With a fish rule and a gluten rule for every pizza added, how many warnings are there? _____ NOTEBOOK Write both rules.

Solution

The left side of → is the **body** of the rule: exactly the description of 1(a). The right side is the **head**: for every answer of the body, a new object *w* is created as the witness of ∃*w*.

```
fishy = a(Pizza)(toppings=a(FishTopping)()).from_(pizzas)
fish_warnings   = inference(AllergenWarning)(pizza=fishy, allergen=Allergen.FISH).tolist()
gluten_warnings = inference(AllergenWarning)(pizza=a(Pizza).from_(pizzas),
allergen=Allergen.GLUTEN).tolist()
```

13 warnings: 5 milk, 1 fish, 7 gluten. Deriving all conclusions in advance is **forward chaining** (materialisation). The predicate of Act 3 works the other way: it derives its conclusion only when a question needs it, which is the strategy of **backward chaining**.

(b) Rules with exceptions. The chef's rule of thumb is a rule tree (Figure 9). PREDICT Write the label of each pizza in the table, then run 4(b).

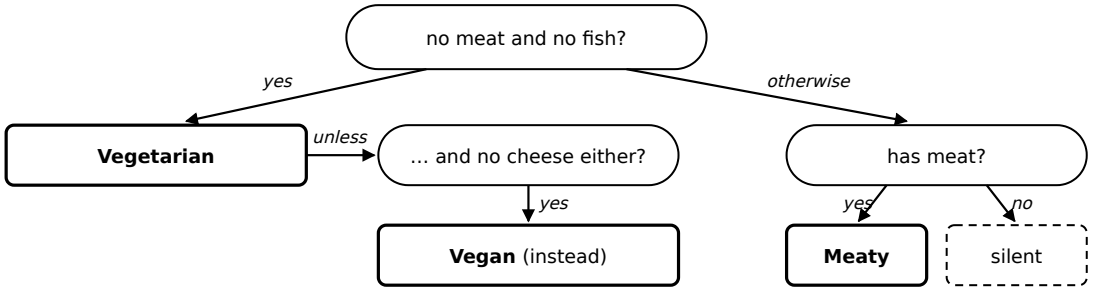


Figure 9. A rule tree: a default conclusion, an exception (*unless*, in EQL `refinement`) and an alternative (*otherwise*, in EQL `alternative`).

Solution

margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed
Vegetarian	**none**	Meaty	Vegetarian	Vegan	Vegan	Vegetarian

napoletana_1 has fish, so the first condition fails; it has no meat, so the alternative fails as well. The rules are **silent**, and silence is not "no": they never concluded "not vegetarian"; reading it so would be one more closed-world step. The *unless* branch is an exception: a more specific conclusion replaces the general one. Rule trees of this shape are known as ripple-down rules.

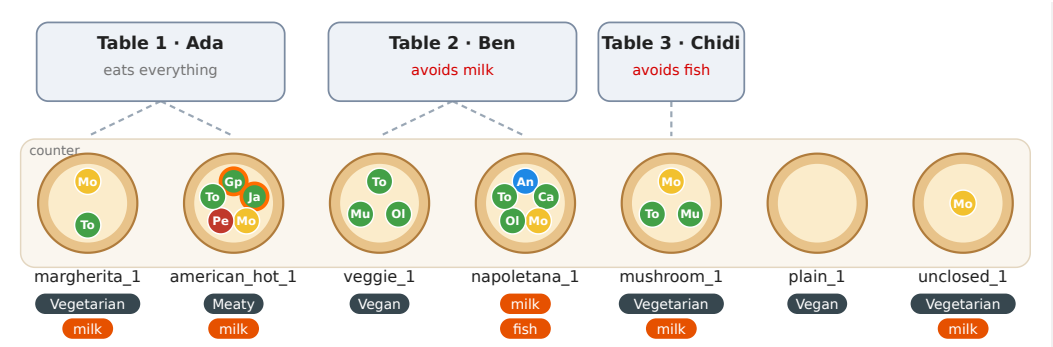


Figure 10. The derived labels, and the milk and fish warnings (gluten, on every pizza, is left out).

(c) The world moves on. *It is busy. The cook mixes up two tickets and grates parmesan over veggie_1, Ben's pizza.* PREDICT What do the three sources below say about veggie_1 now? Which of them protects Ben? Then run 4(c).

Solution

1. the constraint of Act 3, checked now	2. the milk warnings derived in (a)	3. the label of veggie_1, derived again
two violations: napoletana_1 *and* veggie_1	no warning for veggie_1	Vegetarian (it was Vegan)

- Only the constraint protects Ben. It was written before the mistake and is evaluated against the world as it is *now*.
- The stored warnings were derived at a fixed moment and are stale until the rule runs again. Keeping derived facts in line with the facts they were derived from is called **truth maintenance**; a robot, whose world changes all the time, needs it constantly.
- "Vegan" was withdrawn because a fact was *added*. Reasoning in which more knowledge can remove a conclusion is **non-monotonic**; here it is so because the vegan rule relies on the absence of cheese. Classical first-order logic is monotonic: what follows from a set of formulas still follows when formulas are added.

Act 5 · Generating *"A pizza with mozzarella and tomato, please."* 10 min

Dana sits down at table 4 and orders. Her order is a description, exactly like those of Act 1. What is new is who answers it.

A query in EQL is a symbolic object; nothing is computed when it is written. A **backend** is whatever answers it, and the same description can be handed to different backends:

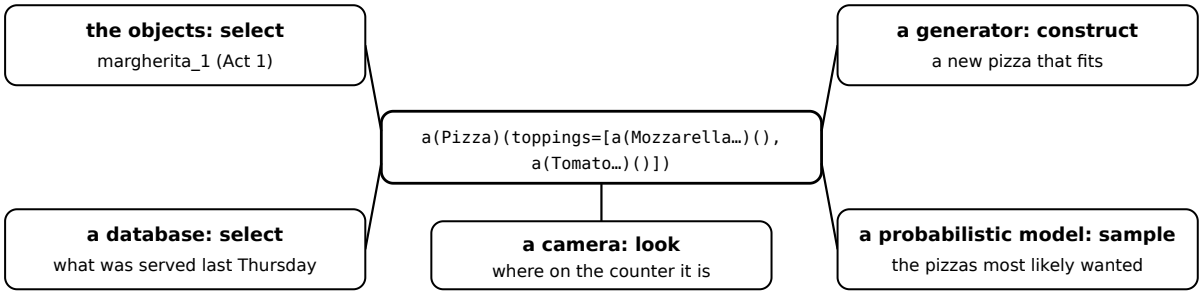


Figure 11. The description is written once, symbolically. Selecting, constructing, sampling and looking are different ways of answering it.

(a) Select or construct? PREDICT What does the counter answer, and what does the kitchen answer? Dana adds "… and a green pepper, I don't mind how hot" (`spiciness=...`, left open). How many pizzas does the kitchen offer? _____ NOTEBOOK Run Act 5.

> **Think.** In Act 1, every answer was an object of the interpretation in Figure 1. What is an answer here? What does it mean, in terms of models, that Dana's open description has several answers?

Solution

The counter (a selecting backend) answers margherita_1; the kitchen (a generative backend) answers a *new* pizza with exactly these toppings. The open description has three answers, one per value of `Spiciness`.

An answer is now a new object that, added to the interpretation, makes the description true: answering means *constructing* a model instead of checking one. An underspecified description can be made true in several ways; the generator enumerates them, a probabilistic backend would rank them by how likely they are.

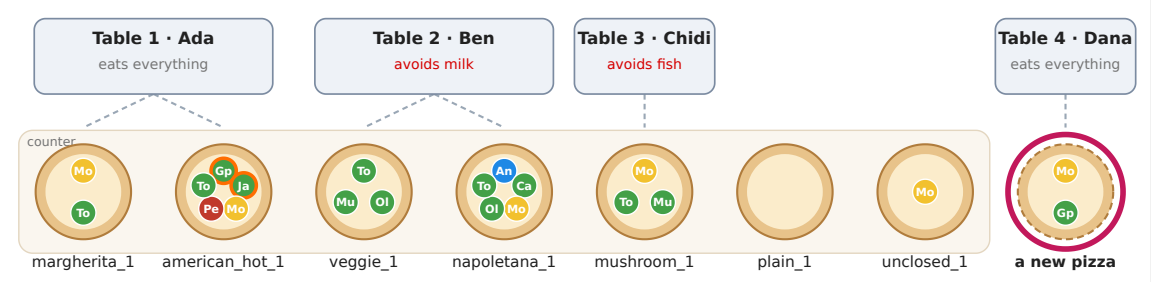


Figure 12. The world has grown: a pizza that did not exist at 19:30.

(b) One plan, many answerers. The robot's next task is a plan with open slots, each written as a description:

> **Bring** ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

PAPER Which kind of backend should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probabilistic model, **G** a generator.

Solution

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made
R and K: IsSafeFor over the pizzas	L: the knowledge base knows *that* it is there, not exactly *where*	K: the orders	S: sampled, then tried	G: Act 5(a)

Several answers are defensible; what matters is that the plan does not say *who* answers. It states what it needs, symbolically, and each slot goes to the backend that can answer that kind of description. The robots of our current research work this way: in a sorting task, the piece to pick up is *looked for* by perception, where it stands is *read off* the robot's digital twin, the hole it fits through is *concluded* by rules, and how to grasp it is *sampled*: one plan, four backends.

Epilogue *The whole picture* 5 min

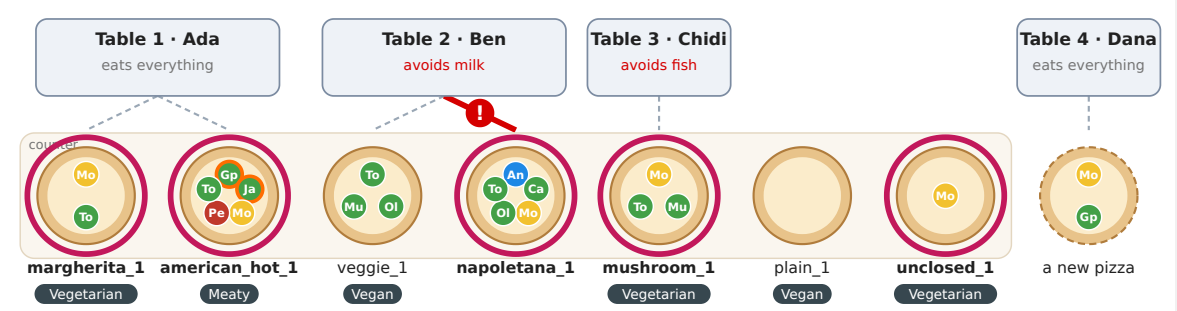


Figure 13. What the robot knows at 20:00: the answers to the chef's question (ringed), the violated constraint, the derived labels, and Dana's new pizza.

(c) Your summary. PAPER For each use of a description, give tonight's example and what it is in logic.

Solution

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?	the cheesy pizzas; who gets which hot topping	the assignments to the free variables that satisfy a formula in one interpretation
2 Describe	Which one is meant?	the pizza with olives and without fish	a definite description: exactly one object satisfies the formula
3 Constrain	Does anything violate it?	no pizza unsafe for its guest	a ∀ checked through its counterexamples ∃¬; an integrity constraint, not an axiom
4 Infer	What follows for each fit?	allergen warnings, menu labels with an exception	a rule body → head, applied by forward chaining; with exceptions, non-monotonic
5 Generate	What could fit?	Dana's pizza; the slots of the plan	constructing new objects that make the description true

Take home. One expression, a description, carried the whole evening. Its meaning is a formula with free variables; what you *do* with it decides whether it is a question, a reference, a constraint, the body of a rule or a specification, and which backend answers it decides whether it is checked, computed or constructed.